

Lecture 5: Data Cleaning & Preprocessing

Week 2: Computational Linguistics

PSYC 51.07: Models of Language and Communication

Learning Objectives

By the end of this lecture, you will be able to:

1. Understand why data cleaning is critical for NLP tasks
2. Apply common preprocessing techniques to raw text
3. Use web scraping to collect text data
4. Implement lemmatization and stemming
5. Make informed decisions about preprocessing strategies

Key theme: Garbage in, garbage out!   

Why Data Cleaning Matters

```
**Real-world text is messy!**
```

```
<div class="columns">
```

Common problems:

- HTML/XML tags: `<p>text</p>`

- Special characters: &
- Inconsistent formatting
- Extra whitespace
- Mixed encodings (UTF-8, ASCII...)
- Typos and misspellings
- Emoji and Unicode 🤗🎉

The Data Cleaning Pipeline

1Raw Text  -> Remove Tags  -> Normalize  -> Tokenize  -> Lemmatize  -> (

****Note:**** Pipeline varies by task! Not all steps are always needed.

Worked Example: Cleaning Messy Text Step by Step



Starting with raw text scraped from a webpage:

```
1raw_text = ""
2<p class="review">I LOVE this    product!!! 🥰
3Check it out at https://example.com/buy
4Contact: support@example.com</p>
5""
```

Step 1: Remove HTML tags

```
1import re
2text = re.sub(r'<.*?>', '', raw_text)
3# Result: "I LOVE this    product!!! 🥰\nCheck it out at https://..."
```

Worked Example: Complete Pipeline

Full transformation:

Stage	Text
Raw	<code><p>I LOVE this product!!! 😍 https://ex.com</p></code>
Remove HTML	<code>I LOVE this product!!! 😍 https://ex.com</code>
Remove URLs	<code>I LOVE this product!!! 😍</code>
Normalize spaces	<code>I LOVE this product!!! 😍</code>
Lowercase	<code>i love this product!!! 😍</code>
Remove extra punctuation	<code>i love this product! 😍</code>

Task-Specific Preprocessing

****Different tasks need different preprocessing:****

| | Emoji | Extra whitespace |

| --- | --- | --- |

| Recognition | Punctuation | Special chars |

| | (preserve style) | (just corrupted text) |

| | | Case (lowercase) |

****Key principle:**** Preserve information relevant to your task! 

Collecting Data: Web Scraping

****Why web scraping?****

– Web = massive source of text data

- News articles, reviews, social media, forums...
- Fresh, diverse, domain-specific content

Popular tools:

- **Beautiful Soup:** Parse HTML/XML, extract text
- **Requests:** Fetch web pages
- **Scrapy:** Full-featured web scraping framework
- **Selenium:** For JavaScript-heavy sites

 **Important considerations:**

Web Scraping with BeautifulSoup

```
**Basic example:**
```

```
```python
```

```
from bs4 import BeautifulSoup
import requests
```

## Fetch webpage

```
url = "https://example.com/article"
response = requests.get(url)
html_content = response.content
```

## Parse HTML

## Get clean article text

```
21---
22
23# Encoding Issues: Concrete Example 🔧
24
25**Common scenario: Web scraping with mixed encodings**
26
```

...continued

```
import chardet
```

## Byte string with unknown encoding

```
raw_bytes = b'Caf\xe9 au lait \x96 delicious!'
```

## Detect encoding

# "Café au lait – delicious!"

```
21 **3. Punctuation Handling**
22 – Keep for sentiment: "Great!" vs. "Great"
23– Remove for topic modeling
24
25
26
27 **4. Case Normalization**
28 – Lowercase: "Apple" and "apple" → same token
29– Preserve for NER: "Apple Inc." vs. "apple fruit"
30
31
32
33 **5. Special Characters**
34 – URLs, email addresses, numbers
35– Replace or remove based on task
36
37
38---
```

# "Café au lait – delicious!"

```
41
42
```

...continued

```
import re
```

```
def preprocess_text(text):
```

```
Remove URLs
```

```
text = re.sub(r'http\S+|www.\S+', '', text)
```

```
Remove email addresses
```

```
text = re.sub(r'\S+@\S+', '', text)
```

```
Remove HTML tags
```

```
text = re.sub(r'<.*?>', '', text)
```

# Example

```
21<div class="column">
22
23**Lemmatization 🧠**
24 – Uses vocabulary + morphology
25– Slower, more accurate
26– Produces real words
27– Requires POS context
28– Example: "better" → "good"
29
30</div>
31</div>
32
33
34
35 **When to use which?**
36 – Stemming: Speed matters, approximate matching OK
37– Lemmatization: Need interpretable, real words
38
```

# Example

```
41
42# Stemming vs. Lemmatization Examples ⚖️
43
44**Concrete comparison with the same words:**
45
46| Word | Porter Stemmer | Lemmatizer | Notes |
47|-----|-----|-----|-----|
48| ran | ran | run | Lemma recognizes irregular verb |
49| runs | run | run | Both work well |
50| running | run | run | Both work well |
51| runner | runner | runner | Both keep as-is (different word) |
52| better | better | good | Lemma handles irregular adjective |
53| best | best | good | Lemma handles superlative |
54| geese | gees | goose | Stemmer produces non-word! |
55| studies | studi | study | Stemmer produces non-word! |
56| organizing | organ | organize | Stemmer too aggressive! |
57
58**Key insight:** Lemmatization produces real words; stemming can produce fragm
```

# Example

```
61
62# When Stemming Goes Wrong 🤔
63
64**Real-world example of stemming problems:**
65
```

...continued

```
from nltk.stem import PorterStemmer stemmer = PorterStemmer()
```

## Over-stemming: Different words become the same

```
words = ['universe', 'university', 'universal']
stems = [stemmer.stem(w) for w in words]
print(stems) # ['univers', 'univers', 'univers']
```

# yesterday yesterday NOUN

```
21 **Recommendation:** Use spaCy for most tasks!
22
23 It's faster, more accurate, and easier to use in production.
24
25---
26
27# Stop Words Removal 🛑
28
29
30 **What are stop words?**
31 - Common words with little semantic value
32- Examples: "the", "a", "is", "in", "and", "or"
33- Language-specific
34
35
36
37 **When to remove?**
38 - ✅ Topic modeling, text classification
```

# yesterday yesterday NOUN

```
41- ❌ Machine translation
42- ❌ Text generation
43
44
45
46 **Warning:** Modern neural models often don't need stop word removal!
47
48 They can learn to ignore them or use them for grammar.
49
50---
51
52# Stop Words in Practice 🧑‍💼
53
54
```

...continued

```
from nltk.corpus import stopwords import spacy
```

# Output: ['example', 'showing', 'stop', 'word', 'removal']

```
21- ❌ Remove: URLs, HTML tags
22- 🤔 Maybe: Extra repeated punctuation (!!!)
23
24
25
26 **Result:**
27
28 *"I LOVE this product! 😍 Best purchase ever!"*
29
30
31 Preserves emotion and intensity!
32
33---
34
35# Discussion: Preprocessing Trade-offs 🤔
36
37
38 **Questions to consider:**
```

## Output: ['example', 'showing', 'stop', 'word', 'removal']

```
41 - Think: "US" (United States) vs. "us" (pronoun)
422. **Task dependency:** Why does preprocessing differ by task?
43 - Hint: What signals matter for sentiment vs. topic modeling?
443. **Modern models:** Do transformer models still need heavy preprocessing?
45 - Consider: BERT handles subwords, capitalization, punctuation...
464. **Bias introduction:** Can preprocessing introduce bias?
47 - Example: Removing slang might remove cultural markers
48
49
50---
51
52# Practical Tips for Data Cleaning 💡
53
54
55 **Best practices:**
56
57 1. **Inspect your data first!** 👁️
58 - Look at samples before deciding on preprocessing
```

# Output: ['example', 'showing', 'stop', 'word', 'removal']

```
613. **Document your pipeline** 
62 - Track what preprocessing you applied and why
634. **Experiment!** 
64 - Try different approaches, measure impact on task
655. **Validate incrementally** 
66 - Check results after each preprocessing step
676. **Consider automation** 
68 - Use libraries: spaCy, NLTK, HuggingFace tokenizers
69
70
71---
72
73# Complete Preprocessing Pipeline Example 
74
75
```

...continued